

PRACTICAL GUIDE

Lakeflow Connect

Ingesting Azure SQL Database with the managed SQL Server connector

This guide ingests an Azure SQL Database into Unity Catalog with the managed SQL Server connector in Lakeflow Connect. The connector is made of three objects. A **Unity Catalog connection** stores the host and credentials of the source. An **ingestion gateway** is a pipeline on classic compute that runs continuously, reads an initial snapshot and then the change feed from the database, and stages the data in a Unity Catalog volume. A **managed ingestion pipeline** runs on serverless compute and applies the staged data to streaming tables in your destination schema.

The source in this course is the `AdventureWorksLT` sample database, created by `infra/terraform` in the course repository and prepared by `infra/sql/enable_cdc.sql`. The demo notebook `02a_lakeflow_connect_demo` follows the same steps in class.

Before you start, you need:

- an Azure subscription where you may create resources, with the Contributor and User Access Administrator roles, plus Terraform 1.5 or later, the Azure CLI and `sqlcmd` (or Azure Data Studio)
- an Azure Databricks workspace with Unity Catalog and serverless compute enabled
- permission to create clusters without restrictions, because the gateway runs on classic compute
- in Unity Catalog: `CREATE CONNECTION` on the metastore (or `USE CONNECTION` on an existing connection), `USE CATALOG` on the target catalog, and either `USE SCHEMA`, `CREATE TABLE` and `CREATE VOLUME` on existing schemas or `CREATE SCHEMA` on the catalog

The training subscription is shared with other courses. Agree on a time window before you run `terraform apply` or `terraform destroy`.

Step	Task
1	Provision the source database with Terraform
2	Enable change data capture and create the ingestion user
3	Create the Unity Catalog connection
4	Create the gateway and the ingestion pipeline
5	Verify the landed tables
6	Test the change data capture round trip
7	Control costs and tear down

STEP 1

Provision the source database with Terraform

In `infra/terraform`, copy the example variables file and edit it. Set the SQL administrator password through an environment variable rather than in the file, and add the public IP address of the computer you will run `sqlcmd` from.

```
cd infra/terraform
cp terraform.tfvars.example terraform.tfvars      # prefix, dates, trainer_ip_cidrs
export TF_VAR_sql_admin_password='<strong password>'
terraform init
terraform plan
terraform apply
```

The file `sql_server.tf` creates the following resources for Lakeflow Connect:

Resource	Setting	Why
azurerm_mssql_server	TLS 1.2, administrator deatrainner	Logical server that hosts the database
azurerm_mssql_database	sku_name = "GP_S_Gen5_1", min_capacity = 0.5, auto_pause_delay_in_minutes = 60, sample_name = "AdventureWorksLT"	General Purpose serverless with up to 1 vCore: the cheapest tier that supports CDC; it pauses after an hour without activity
azurerm_mssql_firewall_rule AllowAzureServices	0.0.0.0 to 0.0.0.0	Lets the gateway, which runs in Azure, reach the server
azurerm_mssql_firewall_rule trainer-NN	From trainer_ip_cidrs	Lets you connect with sqlcmd from your own computer

CDC on Azure SQL Database is available in every vCore tier, but in the DTU model only from S3 upwards, so do not lower the SKU. The *Allow Azure services* rule opens the server to any Azure tenant and is acceptable only for a short-lived training database. In production, use a VNet-injected workspace with a service endpoint or a private endpoint.

When apply finishes, read the values you need in the next steps:

```
terraform output sql_server_fqdn          # sql-dea-training-
<suffix>.database.windows.net
terraform output sql_database_name        # AdventureWorksLT
terraform output lakeflow_connect_connection_summary # host, port 1433, database, user
```

STEP 2

Enable change data capture and create the ingestion user

The script `infra/sql/enable_cdc.sql` enables CDC for the database and for four tables in the `SalesLT` schema: `Customer`, `Product`, `SalesOrderHeader` and `SalesOrderDetail`. It also creates a contained database user, `lakeflow_connect`, with read access to these tables and to the `cdc` schema, plus the metadata permissions the connect- or needs on Azure SQL Database (`VIEW DATABASE STATE`, `SELECT` on the sys catalog views it reads, `EXECUTE` on `sp_tables`, `sp_columns_100`, `sp_pkeys` and `sp_statistics_100`). The script checks what already exists, so you can run it again safely.

Before running it, replace the placeholder password `CHANGE_ME-LakeflowC0nnect!` in the `CREATE USER` statement with your own strong password. You will enter the same password in the connection in step 3. Then run the script against the `AdventureWorksLT` database, not `master`, as the SQL administrator:

```
sqlcmd -S "$(terraform output -raw sql_server_fqdn)" -d AdventureWorksLT \  
-U deatrainr -P "$TF_VAR_sql_admin_password" -i ../sql/enable_cdc.sql
```

If you already ran the script with the placeholder password, change it:

```
ALTER USER lakeflow_connect WITH PASSWORD = N'<your password>';
```

Check the result. The first query should return `1`, and the procedure should list four capture instances:

```
SELECT is_cdc_enabled FROM sys.databases WHERE name = DB_NAME();  
EXEC sys.sp_cdc_help_change_data_capture;
```

Change tracking or CDC

The connector reads changes with either SQL Server change tracking or change data capture. If both are enabled on a table, it uses change tracking.

	Change tracking	Change data capture
Records	That a row changed	Every operation, with before and after images
Requires	A primary key	No primary key needed
Load on the source	Low	Higher: reads the transaction log and stores change tables
Azure SQL tier	Any	vCore tiers, or DTU S3 and higher
Databricks recommendation	Tables with a primary key	Tables without a primary key

The four `SalesLT` tables have primary keys, so change tracking would also work. The course uses CDC so that you can see the change tables in the database. The alternative is commented out at the end of `enable_cdc.sql`. Enable only one mechanism per table.

Additional permissions

`enable_cdc.sql` does not create the objects that track schema changes (DDL support objects). In step 4, the wizard's **Validate** button reports anything that is missing. The most reliable fix is the Databricks utility script, `utility_script.sql`, linked from the connector documentation page *Prepare SQL Server for ingestion using the utility objects script*. Run it as the administrator in `AdventureWorksLT`, and then run:

```
EXEC dbo.lakeflowSetupChangeDataCapture @Tables = 'SCHEMAS:SalesLT', @User = 'lakeflow_connect';  
EXEC dbo.lakeflowFixPermissions @Tables = 'SCHEMAS:SalesLT', @User = 'lakeflow_connect';
```

STEP 3

Create the Unity Catalog connection

In the Databricks workspace, open **Catalog**, then **Connect** (labelled **External data** in some workspaces), then **Connections**, and select **Create connection**. You can also create the connection from the ingestion wizard in step 4.

Field	Value
Connection name	retailhub_sqlserver
Connection type	SQL Server
Host	Output sql_server_fqdn, for example sql-dea-training-ab12c.database.windows.net
Port	1433
User	lakeflow_connect
Password	The password you set in step 2

Select **Create connection**. The connection is now a Unity Catalog securable: users who create ingestion pipelines need `USE CONNECTION` on it, but never see the password.

```
SHOW CONNECTIONS;  
DESCRIBE CONNECTION retailhub_sqlserver;  
GRANT USE CONNECTION ON CONNECTION retailhub_sqlserver TO `data_engineers`;
```

STEP 4

Create the gateway and the ingestion pipeline

In the workspace sidebar, open **Data Ingestion** and, under **Databricks connectors**, select **SQL Server**. The wizard creates both pipelines.

Wizard page	Setting	Value in this course
Connection	Connection	retailhub_sqlserver
Ingestion setup	Ingestion pipeline name	retailhub_sqlserver_ingestion
	Event log location	retailhub_trainer.lakeflow_staging
	Ingestion gateway name	retailhub_sqlserver_gateway
	Staging location	Catalog retailhub_trainer, schema lakeflow_staging; the gateway creates its volume here
Source	Tables	AdventureWorksLT › SalesLT › Customer, Product, SalesOrderHeader, SalesOrderDetail
	History tracking	Off (SCD type 1): target rows are updated and deleted like the source
Destination	Catalog and schema	retailhub_trainer.lakeflow_connect
Database setup	Validate	Fix anything reported (see step 2) before continuing
Schedules and notifications	Schedule	None for the demo; in production, for example every 30 minutes
	Notifications	Your e-mail on failure

Select **Save and run pipeline**. In **Jobs & Pipelines** you now see two pipelines:

- The **gateway** starts classic compute, takes an initial snapshot of the four tables into the staging volume, and then keeps reading the change feed. It is designed to run continuously.
- The **ingestion pipeline** runs as triggered updates on serverless compute. Each update applies the staged snapshot and changes to the destination streaming tables.

The first snapshot takes a few minutes. The list view of the ingestion pipeline shows how many records each table has processed; the *Upserted records* and *Deleted records* columns can be added through the column settings. One pipeline can ingest up to 250 tables, and each ingestion pipeline is paired with exactly one gateway.

Give the course participants read access to the destination:

```
GRANT USE CATALOG ON CATALOG retailhub_trainer TO `dea-training`;
GRANT USE SCHEMA, SELECT ON SCHEMA retailhub_trainer.lakeflow_connect TO `dea-training`;
```

STEP 5

Verify the landed tables

Run the following queries in a notebook or in the SQL editor after the first update of the ingestion pipeline has finished. Unity Catalog stores table names in lower case, so `SalesOrderHeader` becomes `salesorderheader`.

```
SHOW TABLES IN retailhub_trainer.lakeflow_connect;

SELECT 'customer' AS table_name, COUNT(*) AS row_count FROM
retailhub_trainer.lakeflow_connect.customer
UNION ALL SELECT 'product', COUNT(*) FROM retailhub_trainer.lakeflow_connect.product
UNION ALL SELECT 'salesorderheader', COUNT(*) FROM
retailhub_trainer.lakeflow_connect.salesorderheader
UNION ALL SELECT 'salesorderdetail', COUNT(*) FROM
retailhub_trainer.lakeflow_connect.salesorderdetail;
```

```
-- Top products by wholesale revenue
SELECT p.Name AS product_name, SUM(d.OrderQty) AS units, ROUND(SUM(d.LineTotal), 2) AS revenue
FROM retailhub_trainer.lakeflow_connect.salesorderdetail d
JOIN retailhub_trainer.lakeflow_connect.product p ON d.ProductID = p.ProductID
GROUP BY p.Name
ORDER BY revenue DESC
LIMIT 10;
```

```
DESCRIBE TABLE EXTENDED retailhub_trainer.lakeflow_connect.customer; -- streaming table owned by
the pipeline
DESCRIBE HISTORY retailhub_trainer.lakeflow_connect.customer LIMIT 5;
```

Compare the row counts with `SELECT COUNT(*)` on the source tables in Azure SQL. The data types are converted automatically: for example, `MONEY` becomes `DECIMAL(19,4)`, `DATETIME` becomes `TIMESTAMP`, and `UNIQUEIDENTIFIER` and `XML` become `STRING`.

STEP 6

Test the change data capture round trip

Run a query in Databricks to record the current value:

```
SELECT CustomerID, FirstName, LastName, Phone, ModifiedDate
FROM retailhub_trainer.lakeflow_connect.customer
WHERE CustomerID = 1;
```

Change the row in Azure SQL, using the Query editor in the Azure portal or `sqlcmd`, and confirm that CDC captured the change. On Azure SQL Database the capture job runs about every 20 seconds.

```

UPDATE SalesLT.Customer
SET    Phone = '555-TRAINING-42', ModifiedDate = SYSDATETIME()
WHERE  CustomerID = 1;

SELECT __$operation, CustomerID, Phone
FROM   cdc.SalesLT_Customer_CT
WHERE  CustomerID = 1;          -- operation 3 = before update, 4 = after update

```

In **Jobs & Pipelines**, open the ingestion pipeline and select **Start**. The gateway has already staged the change, and the update merges it into the destination table. Run the Databricks query again: **Phone** now shows **555-TRAINING-42**. Inserts and deletes in the source reach the destination in the same way; with history tracking off, a deleted source row is removed from the target.

STEP 7

Control costs and tear down

Component	What you pay for	How to limit it
Ingestion gateway	Classic VMs plus DBUs for as long as it runs	The largest cost in this setup; delete it right after the demo
Ingestion pipeline	Serverless DBUs per update	Run it manually or on an infrequent schedule
Azure SQL Database	vCore-seconds while resumed, plus storage	Auto-pause after 60 idle minutes; a running gateway keeps the database active, and CDC does not capture changes while the database is paused
Staging volume and tables	Storage	Drop the schemas when you are finished

The Databricks documentation warns against stopping the gateway in production. If it stays stopped longer than the change retention period in the source (three days by default for CDC), changes are lost and the tables need a full refresh. For a short training this does not matter, so remove everything as soon as the demo is over:

1. In **Jobs & Pipelines**, delete the ingestion pipeline and then the gateway.
2. Drop the Databricks objects that the pipelines and you created:

```

DROP SCHEMA IF EXISTS retailhub_trainer.lakeflow_connect CASCADE;
DROP SCHEMA IF EXISTS retailhub_trainer.lakeflow_staging CASCADE;
DROP CONNECTION IF EXISTS retailhub_sqlserver;

```

3. After the last training day, remove the Azure resources and confirm that the resource group is gone:

```

cd infra/terraform
terraform destroy
az group list --table | grep dea

```

Summary

Where	Step	Result
Terminal, Terraform	1. <code>terraform apply</code>	Azure SQL server, serverless AdventureWorksLT database, firewall rules, outputs
sqlcmd	2. <code>enable_cdc.sql</code> with your own password	CDC on four SalesLT tables, user <code>lakeflow_connect</code>
Catalog, Connections	3. Create connection	Unity Catalog connection <code>retailhub_sqlserver</code>
Data Ingestion, SQL Server	4. Wizard	Gateway, staging volume, ingestion pipeline, grants for participants
SQL editor or notebook	5. Verification queries	Four streaming tables in <code>retailhub_trainer.lakeflow_connect</code>
Azure SQL and Jobs & Pipelines	6. <code>UPDATE</code> , then Start	The change appears in the destination table
Jobs & Pipelines, Terraform	7. Delete, drop, <code>terraform destroy</code>	No resources left

Troubleshooting

Symptom	Cause and fix
sqlcmd login timeout	Your IP address is not in <code>trainer_ip_cidrs</code> (check with <code>curl -s ifconfig.me</code> , add it and apply again), or the serverless database is resuming from pause; retry after about a minute
Connection test or gateway cannot reach the server	The <code>AllowAzureServices</code> firewall rule is missing, or the host or port is wrong; the host must be the full <code>*.database.windows.net</code> name and the port 1433
The first connection after a quiet period is slow or times out	The database was auto-paused; resuming takes 30 to 60 seconds; run a query shortly before the demo
sp_cdc_enable_db fails	You are connected to <code>master</code> (use <code>-d AdventureWorksLT</code>), or the tier is below 1 vCore or S3; keep <code>GP_S_Gen5_1</code> or switch to change tracking
Validation reports that CDC or change tracking is not enabled, or permissions are missing	Run <code>enable_cdc.sql</code> again; check <code>SELECT is_tracked_by_cdc FROM sys.tables</code> ; run the Databricks utility script with <code>lakeflowFixPermissions</code>
<code>PERMISSION_DENIED: You are not authorized to create clusters</code>	The user lacks the unrestricted cluster creation entitlement that the gateway needs; a workspace admin must grant it
The wizard does not allow creating the connection, volume or tables	Unity Catalog privileges are missing: <code>CREATE CONNECTION</code> , <code>USE CATALOG</code> , <code>CREATE SCHEMA</code> or <code>USE SCHEMA</code> , <code>CREATE TABLE</code> , <code>CREATE VOLUME</code>
Queries report that the table does not exist	Use lower-case names (<code>salesorderheader</code>) and the destination schema, not the staging schema; wait for the first update to finish
Cannot have multiple queries named ...	Two source schemas contain tables with the same name and target the same destination schema; set different destination names or use separate gateway and pipeline pairs
A source change does not appear	The capture job has not run yet (about 20 seconds), no ingestion update ran after the change, or the database was paused
<code>NON_INCREMENTAL_SCHEMA_CHANGE</code> after adding a column in the source	Run a full refresh of the affected table from the pipeline page